📚 Advanced Java Programming — Comprehensive Study Notes

📋 Table of Contents

- Unit 1: Introduction to Java
  - Topic 1.1: Introduction to Java
  - Topic 1.2: Inheritance
  - Topic 1.3: Exception Handling
  - Topic 1.4: Multithreading
  - Topic 1.5: Applet Programming
  - Topic 1.6: Connecting to a Server
  - Topic 1.7: Implementing Servers
  - Topic 1.8: Making URL Connections
  - Topic 1.9: Socket Programming
  - Topic 1.10: JMS and Message Queues
  - Topic 1.11: Persistence
  - Topic 1.12: Applet vs Application
- Unit 2: JavaBean, Session Beans, Entity Bean, Servlet
  - Topic 2.1: JavaBean
  - Topic 2.2: Java Bean Properties
  - Topic 2.3: Types of Beans
  - Topic 2.4: Introspection
  - Topic 2.5: Stateful Session Bean
  - Topic 2.6: Stateless Session Bean
  - Topic 2.7: Entity Bean
  - Topic 2.8: Servlet Overview and Architecture
  - Topic 2.9: Interface Servlet and the Servlet Life Cycle
  - Topic 2.10: Handling HTTP GET Requests
  - Topic 2.11: Handling HTTP POST Requests
  - Topic 2.12: Session Tracking
  - Topic 2.13: Cookies
  - Topic 2.14: EJB Container Services

- 📊 Coverage Statistics

Unit 1: Introduction to Java, Inheritance, Exception Handling, Multithreading, Applet Programming, Connecting to a Server, Implementing Servers, Making URL Connections, Socket Programming

Topic 1.1: Introduction to Java

Overview

Java is a high-level, class-based, object-oriented programming language developed by Sun Microsystems (now owned by Oracle Corporation). It is platform-independent, meaning Java programs can run on any device with the Java Virtual Machine (JVM) installed.

Key Definitions

> ***Java Virtual Machine (JVM):*** *A virtual machine that enables Java bytecode to be executed on any platform. It acts as an abstract computer that runs Java bytecode.*

> ***Bytecode:*** *Platform-independent compiled Java code with .class extension. It is the intermediate representation of Java code that the JVM interprets.*

> ***WORA (Write Once, Run Anywhere):*** *The principle that Java code written on one platform can run on any other platform that has a JVM without recompilation.*

Java Key Features

Feature	Description
Platform Independent	Works on "write once, run anywhere" principle. Once you write Java code, it can run on any platform with Java installed.
Object-Oriented	Everything in Java is an object. Supports OOP concepts: Object, Class, Inheritance, Polymorphism, Abstraction, Encapsulation.
Robust	Strong memory management system. Helps eliminate errors through compile-time and runtime checking.
Secure	Does not use explicit pointers and runs programs inside a sandbox to prevent untrusted activities.
Multi-threaded	Supports concurrent execution of multiple threads for maximum CPU utilization.
Portable	Java programs can be easily moved from one computer system to another.

Java Editions

- **JSE (Java Standard Edition):** Core Java for standalone desktop applications
- **JEE (Java Enterprise Edition):** Advanced Java for enterprise-level distributed applications, web services, and server-side development
- **JME (Java Micro Edition):** For mobile devices and embedded systems

Basic Syntax Rules

```java
public class MyFirstJavaProgram {
    public static void main(String []args) {
        System.out.println("Hello World");
    }
}
```

Key Takeaways

- ✅ Java provides platform independence through JVM
- ✅ JSE is the foundation; JEE extends it for enterprise applications

- ✅ JME targets mobile and embedded devices
- ✅ Java uses bytecode for platform independence

Topic 1.2: Inheritance

Overview

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a new class to inherit properties and behavior from an existing class.

Key Definitions

> ***Inheritance:*** *A mechanism where a class acquires the properties and behaviors of another class, promoting code reuse.*

> ***Superclass (Parent Class):*** *The class whose properties and methods are inherited by another class.*

> ***Subclass (Child Class):*** *The class that inherits properties and methods from another class.*

> ***Polymorphism:*** *The ability of objects to take on many forms; allows writing code that deals with general object types.*

Types of Inheritance

1. **Single Inheritance:** One parent, one child
2. **Multilevel Inheritance:** Grandparent to Parent to Child
3. **Hierarchical Inheritance:** One parent, multiple children
4. **Multiple Inheritance:** Not supported directly (use interfaces)

Code Example

```java
class Vehicle {
    void move() {
        System.out.println("Vehicle is moving");
    }
}

class Car extends Vehicle {
    void accelerate() {
        System.out.println("Car is accelerating");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.move();        // Output: Vehicle is moving
        car.accelerate();  // Output: Car is accelerating
    }
}
```

Key Takeaways

- ✅ Inheritance enables "is-a" relationship
- ✅ Single inheritance only in Java (use interfaces for multiple)
- ✅ Use `super()` to access parent class methods and constructors
- ✅ Methods marked final cannot be overridden

Topic 1.3: Exception Handling

Overview

Exception handling is a mechanism provided by Java to handle runtime errors in a structured and graceful manner.

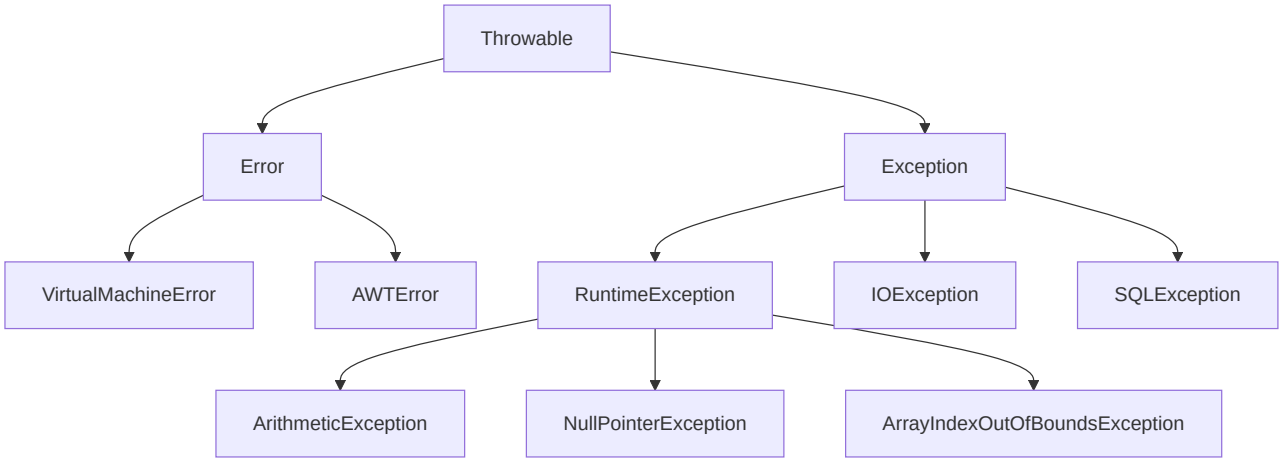
Key Definitions

> ***Exception:*** *An event that disrupts the normal flow of the program's execution.*

> ***Throwable:*** *The root class of the exception hierarchy in Java.*

> ***try-catch:*** *A block of code where exceptions are handled.*

Exception Hierarchy



Types of Exceptions

Type	Description
Checked Exceptions	Compiled-time exceptions that must be handled or declared
Unchecked Exceptions	Runtime exceptions not checked at compile time

Exception Handling Constructs

1. **try:** Block surrounding program statements to monitor for exceptions
2. **catch:** Catches specific kinds of exceptions and handles them
3. **finally:** Code that must execute whether or not an exception occurs
4. **throw:** Used to throw a specific exception from the program
5. **throws:** Specifies which exceptions a method can throw

Syntax

```
try {
    // Code that may throw an exception
} catch (ExceptionType1 e1) {
    // Handle ExceptionType1
} catch (ExceptionType2 e2) {
    // Handle ExceptionType2
} finally {
    // Code that always executes
}
```

Key Takeaways

- ✅ Checked exceptions must be handled or declared with throws
- ✅ Unchecked exceptions occur at runtime
- ✅ Multiple catch blocks must order from specific to general
- ✅ finally block always executes (except System.exit())

Topic 1.4: Multithreading

Overview

Multithreading is a programming concept that allows multiple threads to execute concurrently within a single process.

Key Definitions

> ***Thread:*** *A lightweight subprocess, the smallest unit of processing that can be scheduled by the operating system.*

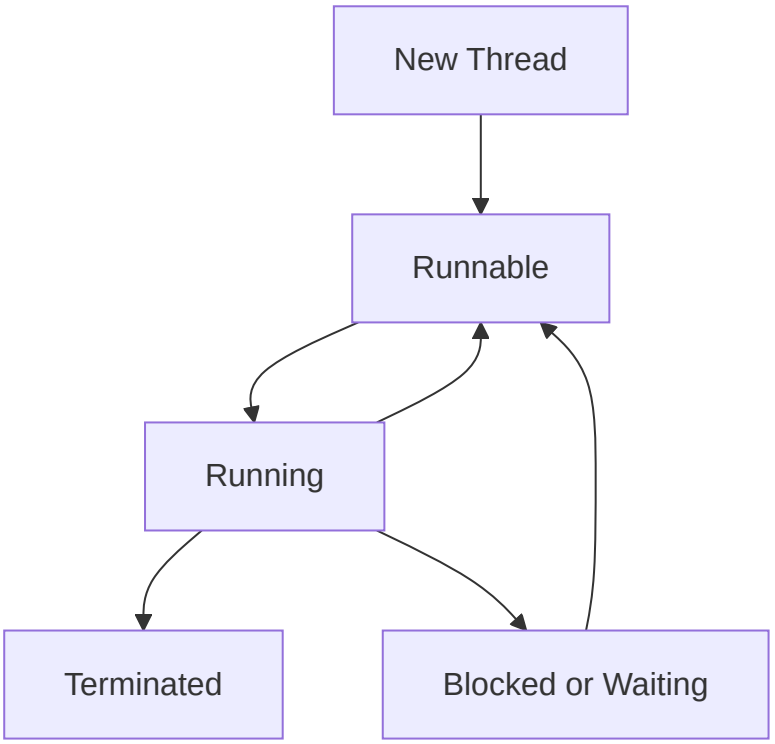
> ***Process:*** *An independent program execution that has its own memory space.*

> ***Runnable Interface:*** *An interface that provides a way to create a thread without extending the Thread class.*

Thread Lifecycle States

1. **New:** Thread object created but not started
2. **Runnable:** Thread is ready to run, waiting for CPU
3. **Running:** Thread is executing
4. **Blocked/Waiting:** Thread is waiting for resources or another thread
5. **Terminated:** Thread has completed execution

Visual: Thread Lifecycle



Creating Threads in Java

Method 1: Extending Thread Class

```java
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread: " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start();
    }
}
```

Method 2: Implementing Runnable Interface

```java
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable: " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start();
    }
}
```

Thread Lifecycle Methods Summary

Method	Description
start()	Causes the thread to begin execution
run()	Contains the code to be executed in the thread
sleep(long ms)	Pauses thread execution for specified milliseconds
join()	Waits for thread to complete execution
wait()	Causes thread to wait until notified
notify()	Wakes up a single waiting thread

Key Takeaways

- ✅ Threads share memory space within a process
- ✅ Runnable interface is preferred over extending Thread
- ✅ Synchronized keyword prevents race conditions
- ✅ Thread lifecycle: New → Runnable → Running → Blocked → Terminated

---

Topic 1.5: Applet Programming

Overview

Applet programming in Java enables developers to create dynamic and interactive content that can be embedded within web pages.

Key Definitions

> ***Applet:*** *A Java program that runs within a web browser. It is embedded in an HTML page.*

> ***Applet Lifecycle:*** *The sequence of methods called from initialization to destruction of an applet.*

Applet Lifecycle Methods

Method	Description
init()	Called once when applet is first loaded
start()	Called after init() and whenever applet restarts
paint(Graphics g)	Called when applet needs to be redrawn
stop()	Called when applet is stopped
destroy()	Called when applet is terminated

Code Examples

Simple Applet

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloWorldApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, World!", 20, 20);
    }
}
```

Drawing Shapes

```java
import java.applet.Applet;
import java.awt.Graphics;

public class ShapeApplet extends Applet {
    public void paint(Graphics g) {
        g.drawRect(20, 20, 100, 50);
        g.drawOval(150, 20, 80, 80);
        g.drawLine(300, 20, 400, 100);
    }
}
```

Animation

```java
import java.applet.Applet;
import java.awt.Graphics;

public class AnimationApplet extends Applet implements Runnable {
    int x = 0;

    public void init() {
        Thread t = new Thread(this);
        t.start();
    }

    public void paint(Graphics g) {
        g.drawString("Moving Text", x, 20);
    }

    public void run() {
        while (true) {
            x += 5;
            if (x > getWidth()) {
                x = 0;
            }
            repaint();
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

HTML APPLET Tag Attributes

Attribute	Description
CODE	Name of the applet class file (required)
WIDTH	Width of applet display area (required)
HEIGHT	Height of applet display area (required)
CODEBASE	Base URL of applet code
NAME	Name for applet instance

Key Takeaways

- ✅ Applets extend Applet or JApplet class
- ✅ No main() method - lifecycle managed by browser/viewer
- ✅ Graphics output via paint() method

Topic 1.6: Connecting to a Server

Overview

Connecting to a server in Java involves network communication using the Socket class.

Steps to Connect to Server

1. **Create a Socket:** Specify server address and port number
2. **Get Input/Output Streams:** For reading and writing data
3. **Communicate:** Exchange data with the server
4. **Close Connection:** Clean up resources

Code Examples

Client Code

```java
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("server_address", port_number);

            OutputStream outputStream = socket.getOutputStream();
            PrintWriter out = new PrintWriter(outputStream, true);

            out.println("Hello, Server!");

            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Server Code

```java
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(port_number);
            Socket clientSocket = serverSocket.accept();

            InputStream inputStream = clientSocket.getInputStream();
            BufferedReader in = new BufferedReader(new InputStreamReader(inputStream));

            String message = in.readLine();
            System.out.println("Message from client: " + message);

            clientSocket.close();
            serverSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Key Takeaways

- ✅ Socket combines IP address and port number for network communication
- ✅ ServerSocket listens on a specific port for incoming connections
- ✅ accept() blocks until client connects

Topic 1.7: Implementing Servers

Overview

Implementing servers in Java involves creating applications that listen for incoming connections from clients.

Server Implementation Steps

1. Create a ServerSocket on a specific port
2. Call accept() to wait for client connections
3. When a client connects, get the Socket object
4. Create input/output streams for communication
5. Process client requests
6. Close connections when done

Code Example

```java
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(12345);
            System.out.println("Server started. Waiting for clients...");

            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client connected: " + clientSocket);

                ClientHandler clientHandler = new ClientHandler(clientSocket);
                clientHandler.start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

class ClientHandler extends Thread {
    private Socket clientSocket;

    public ClientHandler(Socket clientSocket) {
        this.clientSocket = clientSocket;
    }

    public void run() {
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
            PrintWriter out = new PrintWriter(
                clientSocket.getOutputStream(), true);

            String message = in.readLine();
            System.out.println("Message from client: " + message);

            out.println("Server received your message: " + message);
            clientSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Key Takeaways

- ✅ ServerSocket listens on a specific port
- ✅ accept() blocks until client connects

- ✅ Each client gets its own Socket
- ✅ Use threads for multiple concurrent clients

Topic 1.8: Making URL Connections

Overview

Java provides classes for making URL connections and retrieving web resources.

Key Definitions

> ***URL (Uniform Resource Locator):*** *A reference to a web resource that specifies its location on a computer network.*

> ***URLConnection:*** *An abstract class that represents a connection to a URL.*

URL Structure

```
protocol://hostname:port/path?query#fragment
```

Code Example

```java
import java.net.*;
import java.io.*;

public class URLReader {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://example.com");

            URLConnection connection = url.openConnection();

            HttpURLConnection httpConnection = (HttpURLConnection) connection;
            int responseCode = httpConnection.getResponseCode();
            System.out.println("Response Code: " + responseCode);

            BufferedReader reader = new BufferedReader(
                new InputStreamReader(connection.getInputStream()));

            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Key Takeaways

- ✅ URLConnection is abstract; use HttpURLConnection for HTTP
- ✅ getInputStream() for reading response
- ✅ getOutputStream() for sending data (POST)

Topic 1.9: Socket Programming

Overview

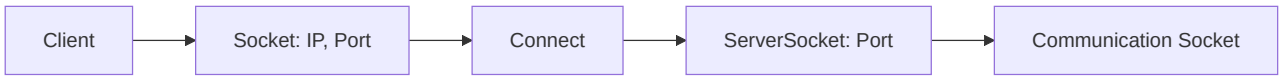
Socket programming enables communication between applications running on different machines over a network.

Key Definitions

> ***Socket:*** *A combination of IP address and port number used for network communication.*

> ***ServerSocket:*** *A socket that listens for incoming client connections on a specific port.*

Visual: Socket Connection Process



Simple Client

```java
import java.net.*;
import java.io.*;

public class Client {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);

            OutputStream outputStream = socket.getOutputStream();
            PrintWriter out = new PrintWriter(outputStream, true);

            out.println("Hello, Server!");

            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Simple Server

```java
import java.net.*;
import java.io.*;

public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server started...");

            Socket clientSocket = serverSocket.accept();
            System.out.println("Client connected");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));

            String message = in.readLine();
            System.out.println("Message: " + message);

            clientSocket.close();
            serverSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Key Takeaways

- ✅ Socket combines IP address and port number
- ✅ Port numbers range from 0 to 65535
- ✅ ServerSocket.accept() blocks until client connects

Topic 1.10: JMS and Message Queues

Overview

JMS (Java Message Service) and queues play crucial roles in messaging systems, facilitating asynchronous communication between distributed applications.

What is JMS?

> ***Java Message Service (JMS):*** *A Java API that provides a standard way for Java applications to create, send, receive, and process messages asynchronously. It abstracts away the complexities of communication protocols.*

Key Concepts of JMS

Concept	Description
Messages	Various types: TextMessage, ObjectMessage, BytesMessage
Producers	Components that create and send messages to destinations
Consumers	Components that receive and process messages
Destinations	Message queues or topics where messages are stored
Connection Factories	Objects used to create connections to JMS providers
Sessions	Transactional context for producing and consuming messages

Message Queue Characteristics

Characteristic	Description
Asynchronous Communication	Producers and consumers operate independently
Reliability	Messages persist until successfully consumed
Load Balancing	Distributes messages across multiple consumers
Guaranteed Delivery	Messages not lost even in system failures
Decoupling	Producers and consumers evolve independently

JMS Messaging Models

1. **Point-to-Point (Queue):** One producer, one consumer
2. **Publish-Subscribe (Topic):** One producer, multiple consumers

Popular Message Queue Implementations

- Apache ActiveMQ
- RabbitMQ
- Apache Kafka
- Amazon Simple Queue Service (SQS)
- IBM MQ

Use Cases

- **Asynchronous Communication:** Sending notifications, event-driven architectures
- **Job Queues:** Managing background tasks, batch processing
- **Integration:** Integrating heterogeneous systems

Key Takeaways

- ✅ JMS provides standard API for messaging
- ✅ Asynchronous communication between components
- ✅ Reliable message delivery guaranteed
- ✅ Point-to-Point and Publish-Subscribe models

Topic 1.11: Persistence

Overview

Persistence refers to the ability of data to outlive the execution of a program. It involves storing and retrieving data from a durable storage medium.

Key Definitions

> ***Persistence:*** *The ability of data to outlive the execution of a program by storing it in a durable storage medium.*

> ***ORM (Object-Relational Mapping):*** *Frameworks that map database entities to object-oriented programming constructs.*

Key Concepts in Persistence

1. **Data Storage:** Storing data in structured format (databases, files, cloud)
2. **CRUD Operations:**
    - Create: Insert new data
    - Read: Retrieve data
    - Update: Modify existing data
    - Delete: Remove data
3. **Object-Relational Mapping (ORM):** Frameworks like Hibernate map Java objects to database tables
4. **Transactions:** Group operations into atomic units (ACID properties)
    - Atomicity: All or nothing
    - Consistency: Valid state
    - Isolation: Concurrent transactions don't interfere
    - Durability: Committed data is permanent
5. **Query Languages:** SQL for relational databases, MongoDB Query Language
6. **Indexing and Optimization:** Improve data retrieval performance

Types of Persistence Mechanisms

Type	Examples	Use Case
Relational Databases	MySQL, PostgreSQL, Oracle	Structured data
NoSQL Databases	MongoDB, Cassandra, Redis	Unstructured/semi-structured
Object Storage	Amazon S3, Google Cloud Storage	Images, videos, documents
File Systems	Local disk, NFS	Simple storage needs
Cloud Storage	AWS S3, Azure Blob	Cloud-native applications

Persistence in Java

```java
// Using JDBC
Connection conn = DriverManager.getConnection(url, user, pass);
PreparedStatement stmt = conn.prepareStatement("INSERT INTO users VALUES (?, ?)");
stmt.setString(1, "John");
stmt.setString(2, "john@example.com");
stmt.executeUpdate();

// Using JPA/Hibernate
@Entity
public class User {
    @Id
    private int id;
    private String name;
    private String email;
}
```

Key Takeaways

- ✅ Persistence enables data storage beyond program execution
- ✅ ORM frameworks simplify database operations
- ✅ ACID properties ensure data integrity
- ✅ Various storage options for different needs

Topic 1.12: Applet vs Application

Overview

Java Applets and Java Applications are both ways to execute Java code, but they differ significantly in deployment and execution context.

Comparison Table

Feature	Java Applet	Java Application
Execution Context	Embedded within a web browser	Executed standalone on client machine
Deployment	Loaded and executed from a web server	Installed and executed locally
User Interaction	Limited interaction within browser window	Full-fledged GUI interaction
Security	Restricted by browser sandbox	Has access to system resources
Network Access	Can make network requests to the server	Can make network requests to external services
Lifespan	Controlled by web browser session	Runs until terminated by user
GUI Components	Limited support	Full support (Swing, JavaFX)
Accessibility	Accessible through URL in web page	Accessed directly from desktop
Deployment Model	Embedded as tag in HTML	Distributed as JAR files or executables
Status	Deprecated due to security issues	Widely used for desktop/enterprise

Detailed Differences

Java Applet

```
<!-- Embedded in HTML -->
<applet code="MyApplet.class" width="400" height="300">
</applet>
```

- Runs inside web browser
- Requires Java plugin
- Security sandbox restrictions
- Used for web-based interactivity
- Now deprecated in modern browsers

Java Application

```
public class MyApp {
    public static void main(String[] args) {
        System.out.println("Application running");
    }
}
```

- Standalone program
- Runs on JVM
- Full system access
- Desktop and enterprise use
- Modern standard for Java programs

Why Applets Became Deprecated

1. **Security Issues:** Many vulnerabilities in browser plugins
2. **Browser Support:** Most browsers removed Java plugin support
3. **Performance:** Slower startup compared to native solutions
4. **Mobile:** Not supported on mobile browsers

Key Takeaways

- ✅ Applets: Browser-based, deprecated
- ✅ Applications: Standalone, widely used
- ✅ Modern Java uses Web Start or applets alternatives

Unit 2: JavaBean, Session Beans, Entity Bean, Servlet

Topic 2.1: JavaBean

Overview

JavaBeans are reusable software components that can be manipulated visually in a builder tool. A JavaBean is a Java class that follows specific conventions.

Key Definitions

> ***JavaBean:*** *A reusable software component that follows specific conventions: no-arg constructor, Serializable, and property access via getter/setter methods.*

> ***Property:*** *A named feature of a JavaBean that can be accessed by users of the object through getter and setter methods.*

Requirements for a Java Bean

1. **Public no-arg constructor:** Must have a no-argument constructor
2. **Serializable:** Should implement `Serializable` interface
3. **Getters and Setters:** Properties accessed via `getXxx()` and `setXxx()` methods
4. **Naming convention:** Boolean properties can use `isXxx()` instead of `getXxx()`

Code Example

```
import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
;
    private String    private int age grade;

    // No-arg constructor
    public Student() { }

    // Parameterized constructor
    public Student(String name, int age, String grade) {
        this.name = name;
        this.age = age;
        this.grade = grade;
    }

    // Getters and Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}
```

Key Takeaways

- ✅ Follow three conventions: no-arg constructor, Serializable, getters/setters
- ✅ JavaBeans are fundamental to EJB and JSP

Topic 2.2: Java Bean Properties

Overview

Java Bean properties define the state of a JavaBean accessed through getter and setter methods.

Property Types

Type	Getter	Setter	Example
Read-Write	`getXxx()`	`setXxx()`	name
Read-Only	`getXxx()`	None	displayName (computed)
Write-Only	None	`setXxx()`	password
Boolean	`isXxx()`	`setXxx()`	active

Code Example

```java
public class PropertyExample {
    private String name;
    private int age;
    private boolean active;
    private String password;

    // Read-Write property
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    // Read-Only property (computed)
    public String getDisplayName() {
        return name + " (" + age + ")";
    }

    // Boolean property
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }

    // Write-Only property
    public void setPassword(String password) {
        this.password = hash(password);
    }
}
```

Key Takeaways

- ✅ Properties expose internal state through controlled access
- ✅ Encapsulation maintained through getter/setter methods
- ✅ Boolean properties use "is" prefix

Topic 2.3: Types of Beans

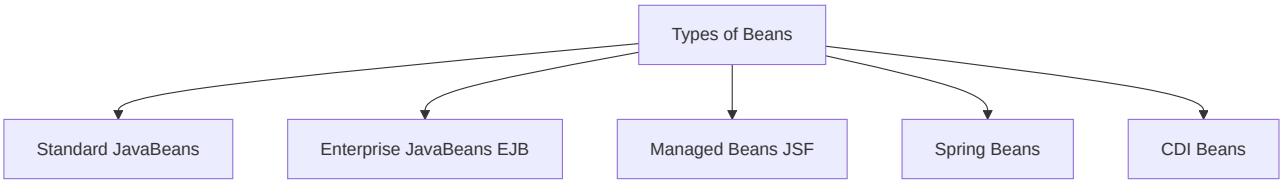
Overview

There are various types of beans in Java enterprise applications, each serving different purposes.

Types of Beans

Type	Description
Standard JavaBeans	Simple POJOs with properties
Enterprise JavaBeans (EJB)	Server-side business components
Managed Beans (JSF)	Backing beans for JSF pages
Spring Beans	Managed by Spring framework
CDI Beans	Contexts and Dependency Injection

Visual: Types of Beans



Key Takeaways

- ✅ Different types of beans serve different architectural purposes
- ✅ Enterprise beans provide enterprise-level services

Topic 2.4: Introspection

Overview

Introspection is a mechanism that allows examination and analysis of an object's properties, methods, and metadata at runtime.

Key Definitions

> ***Introspection:*** *A mechanism that allows programs to inspect and manipulate the structure and behavior of objects dynamically without prior knowledge of their implementation details.*

Ways of Implementing Introspection

1. Java Reflection API

Allows inspection and manipulation of fields, methods, and constructors at runtime.

```
// Get class information
Class<?> beanClass = MyBean.class;

// Get all fields
Field[] fields = beanClass.getDeclaredFields();

// Get all methods
Method[] methods = beanClass.getDeclaredMethods();

// Get specific method
Method getNameMethod = beanClass.getMethod("getName");
Object value = getNameMethod.invoke(beanInstance);
```

2. JavaBeans Introspector

Utility class that simplifies introspection for JavaBean classes.

```java
import java.beans.*;

try {
    BeanInfo beanInfo = Introspector.getBeanInfo(MyBean.class);
    PropertyDescriptor[] pds = beanInfo.getPropertyDescriptors();

    for (PropertyDescriptor pd : pds) {
        System.out.println("Property: " + pd.getName());
        System.out.println("Type: " + pd.getPropertyType());
    }
} catch (IntrospectionException e) {
    e.printStackTrace();
}
```

3. Apache Commons BeanUtils

Library providing utility methods for manipulating JavaBean properties.

```java
import org.apache.commons.beanutils.PropertyUtils;

// Get property value
Object value = PropertyUtils.getProperty(bean, "propertyName");

// Set property value
PropertyUtils.setProperty(bean, "propertyName", value);

// Copy properties
PropertyUtils.copyProperties(targetBean, sourceBean);
```

4. Spring BeanWrapper

Spring Framework interface for introspecting and manipulating bean properties.

```java
import org.springframework.beans.*;

BeanWrapper wrapper = new BeanWrapperImpl(myBean);

// Get property value
Object propertyValue = wrapper.getPropertyValue("propertyName");

// Set property value
wrapper.setPropertyValue("propertyName", newValue);

// Check if property is readable/writable
boolean readable = wrapper.isReadableProperty("name");
boolean writable = wrapper.isWritableProperty("name");
```

Comparison of Introspection Methods

Method	Pros	Cons
Reflection API	Built-in, complete control	Verbose, error-prone
Introspector	Simplified, standard approach	Limited to JavaBeans
Apache Commons	Easy to use, feature-rich	External dependency
Spring BeanWrapper	Framework integration	Spring dependency

Key Takeaways

- ✅ Introspection examines objects at runtime
- ✅ Java Reflection API is built-in
- ✅ Introspector simplifies JavaBean introspection
- ✅ Various libraries provide convenient utilities

Topic 2.5: Stateful Session Bean

Overview

A Stateful Session Bean maintains conversational state across method calls within a session. It is dedicated to a single client.

Key Definitions

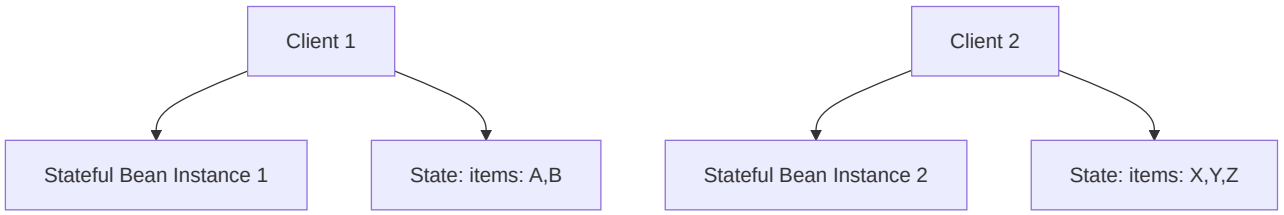
> ***Stateful Session Bean:*** *A session bean that maintains conversational state with the client across multiple method calls.*

Characteristics

- ✅ Maintains client state across multiple requests
- ✅ One bean instance per client

- ✅ Removed when session ends
- ✅ Not persistent
- ✅ Cannot be pooled

Visual: Stateful Session Bean



Code Example

```java
import javax.ejb.Stateful;

@Stateful
public class ShoppingCartBean implements ShoppingCart {
    private List<String> items = new ArrayList<>();
    private String customerId;

    public void setCustomerId(String customerId) {
        this.customerId = customerId;
    }

    public void addItem(String item) {
        items.add(item);
    }

    public void removeItem(String item) {
        items.remove(item);
    }

    public List<String> getItems() {
        return items;
    }

    public void clear() {
        items.clear();
    }
}
```

Key Takeaways

- ✅ Maintains conversational state
- ✅ One-to-one relationship with client

- ✅ Removed when session times out

Topic 2.6: Stateless Session Bean

Overview

A Stateless Session Bean does not maintain conversational state with any client. Each bean instance can be used by any client.

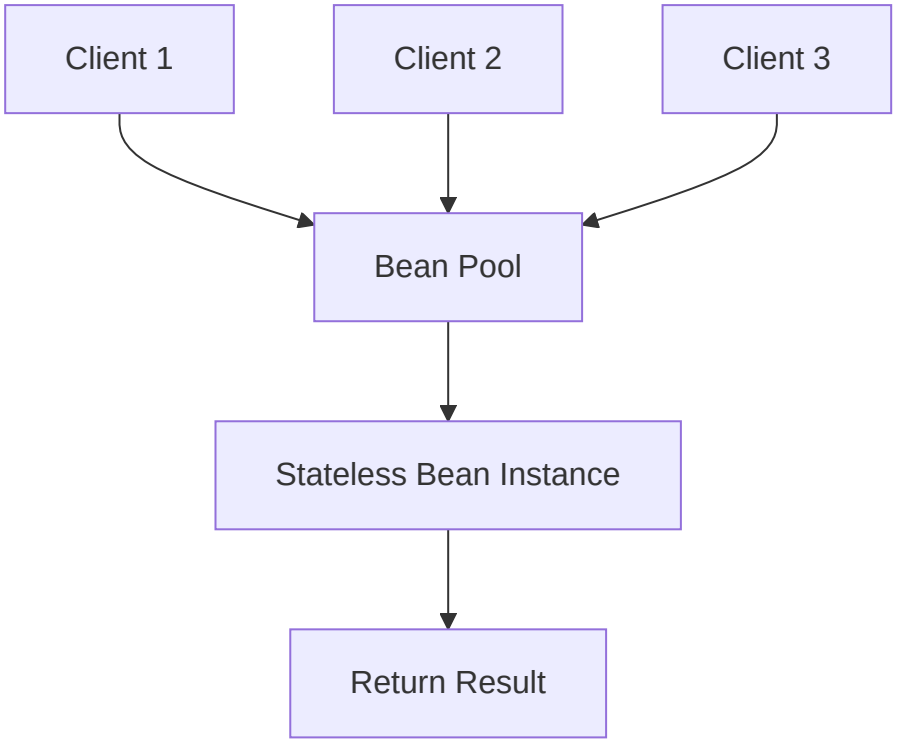
Key Definitions

> ***Stateless Session Bean:*** *A session bean that doesn't maintain conversational state with the client. Provides better scalability.*

Characteristics

- ✅ No conversational state between method calls
- ✅ Any client can use any instance from the pool
- ✅ Better scalability
- ✅ Can implement web services
- ✅ Stateless beans are pooled by the container

Visual: Stateless Session Bean Pool



Code Example

```java
import javax.ejb.Stateless;

@Stateless
public class CalculatorBean implements Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }
}
```

Key Takeaways

- ✅ Don't maintain client-specific state
- ✅ Container manages a pool of instances
- ✅ More scalable than stateful beans

Topic 2.7: Entity Bean

Overview

An Entity Bean represents persistent business data stored in a database.

Key Definitions

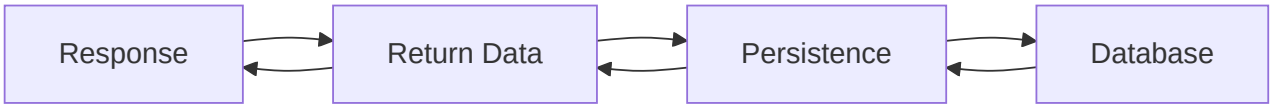
> ***Entity Bean:*** *A server-side component representing persistent business data stored in a database.*

> ***Primary Key:*** *Unique identifier for entity bean instances.*

Entity Bean Types

1. **Bean-Managed Persistence (BMP)**
   - Developer writes database access code explicitly
   - More flexibility but more work
2. **Container-Managed Persistence (CMP)**
   - Container handles database operations automatically
   - Developer focuses on business logic

Visual: Entity Bean with Database



Key Takeaways

- ✅ Represents persistent data in database
- ✅ Has unique primary key
- ✅ Survives server crashes
- ✅ Can be shared by multiple clients
- ⚠️ Now superseded by Java Persistence API (JPA)

Topic 2.8: Servlet Overview and Architecture

Overview

Servlets are Java programs that run on a web server and handle client requests dynamically.

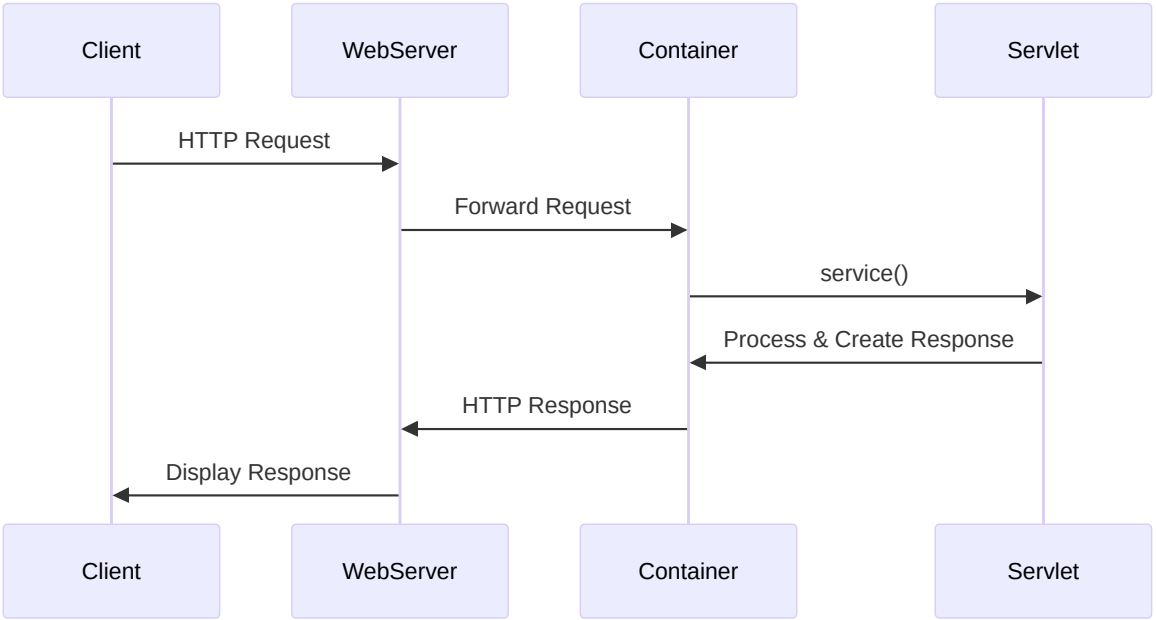
Key Definitions

> ***Servlet:*** *A Java program that runs on a web server, handling HTTP requests and generating dynamic responses.*

Servlet Architecture Components

1. **Client (Web Browser):** Sends HTTP requests
2. **Web Server:** Receives requests
3. **Web Container (Servlet Container):** Manages servlet lifecycle, URL mapping
4. **Servlet:** Processes requests

Servlet Request Flow



Servlet Advantages over CGI

- ✅ Better performance (threads vs processes)
- ✅ Platform-independent
- ✅ Robust (automatic memory management)
- ✅ Secure (sandbox model)
- ✅ Portability

Types of Servlets

1. **Generic
2. **Servlet:** Protocol-independentHttpServlet:** HTTP-specific (most commonly used)

Key Takeaways

- ✅ Servlets handle HTTP requests/responses
- ✅ Run in web container (Tomcat, Glassfish, etc.)
- ✅ More efficient than traditional CGI

Topic 2.9: Interface Servlet and the Servlet Life Cycle

Overview

The Servlet interface defines the contract that all servlets must implement.

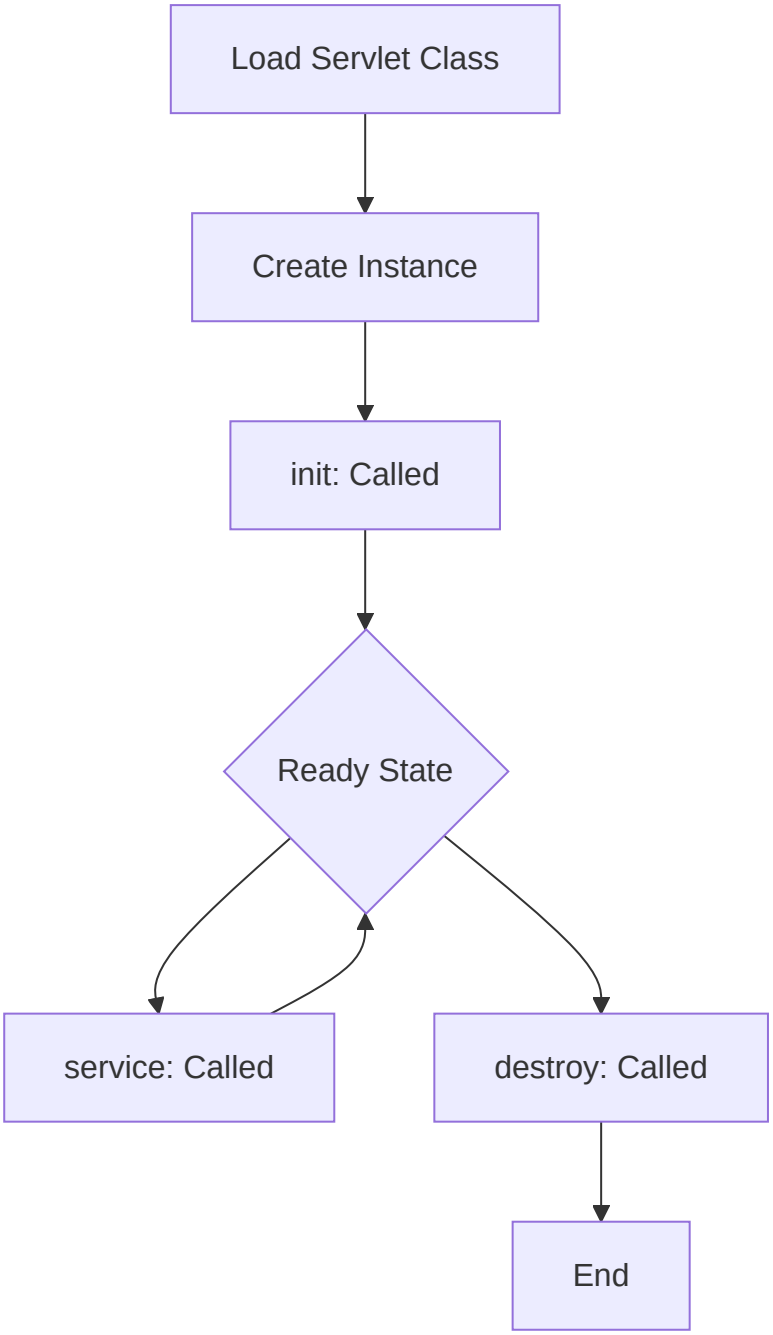
Servlet Interface Methods

Method	Description
init(ServletConfig)	Called once at initialization
service(ServletRequest, ServletResponse)	Called for each request
destroy()	Called when servlet is removed
getServletConfig()	Returns configuration object
getServletInfo()	Returns servlet information

Servlet Life Cycle

1. Servlet class loaded - ClassLoader loads the servlet class
2. Servlet instance created - Container creates instance
3. init() called - Initialization (once)
4. service() called - Handle requests (multiple times)
5. destroy() called - Cleanup before removal

Visual: Servlet Lifecycle



Creating Servlets - Three Ways

1. **Implement Servlet Interface**
2. **Extend GenericServlet**
3. **Extend HttpServlet** (Most common)

Code Example: HttpServlet

```java
import javax.servlet.http.*;
import java.io.*;

public class DemoServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><body>");
        out.println("Welcome to servlet");
        out.println("</body></html>");
    }

    public void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        doGet(req, res);
    }
}
```

Key Takeaways

- ✅ Override doGet() and/or doPost() in HttpServlet
- ✅ init() called once; service() for each request
- ✅ destroy() called when servlet is unloaded

Topic 2.10: Handling HTTP GET Requests

Overview

HTTP GET requests are used to request data from a specified resource.

GET vs POST

GET	POST
Data in URL	Data in request body
Limited data size	Unlimited data size
Can be bookmarked	Cannot be bookmarked
Less secure	More secure

Code Example

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    String username = request.getParameter("username");
    String[] items = request.getParameterValues("items");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    out.println("<html><body>");
    out.println("<h1>GET Request Received</h1>");
    out.println("<p>Username: " + username + "</p>");
    out.println("</body></html>");
}
```

Key Takeaways

- ✅ GET parameters visible in URL
- ✅ Used for retrieving data
- ✅ Can be cached and bookmarked

Topic 2.11: Handling HTTP POST Requests

Overview

HTTP POST requests are used to send data to be processed to a specified resource.

Code Example

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    request.setCharacterEncoding("UTF-8");

    String username = request.getParameter("username");
    String password = request.getParameter("password");
    String email = request.getParameter("email");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<html><body>");
    out.println("<h1>POST Request Received</h1>");
    out.println("<p>Thank you, " + username + "!</p>");
    out.println("</body></html>");
}
```

Key Takeaways

- ✅ POST data hidden from URL
- ✅ Used for submitting sensitive data
- ✅ No size limitations

Topic 2.12: Session Tracking

Overview

HTTP is stateless; session tracking maintains user state across multiple requests.

Methods

Method	Pros	Cons
Cookies	Simple, persistent	Can be disabled
URL Rewriting	Always works	Breaks bookmarking
HttpSession	Server-managed, secure	Server memory
Hidden Fields	Works without cookies	Extra HTML

HttpSession Example

```java
// Creating/Getting session
HttpSession session = request.getSession();

// Setting attributes
session.setAttribute("user", "John");
session.setAttribute("cart", cart);

// Getting attributes
String user = (String) session.getAttribute("user");
List cart = (List) session.getAttribute("cart");

// Removing attributes
session.removeAttribute("user");

// Invalidating session
session.invalidate();
```

Key Takeaways

- ✅ HTTP is stateless - need explicit session tracking
- ✅ HttpSession is the most common approach
- ✅ Sessions have configurable timeout

Topic 2.13: Cookies

Overview

Cookies are key-value pairs stored on the client side for maintaining state.

Types of Cookies

Type	Description
Session Cookie	Deleted when browser closes
Persistent Cookie	Has expiration date

Code Examples

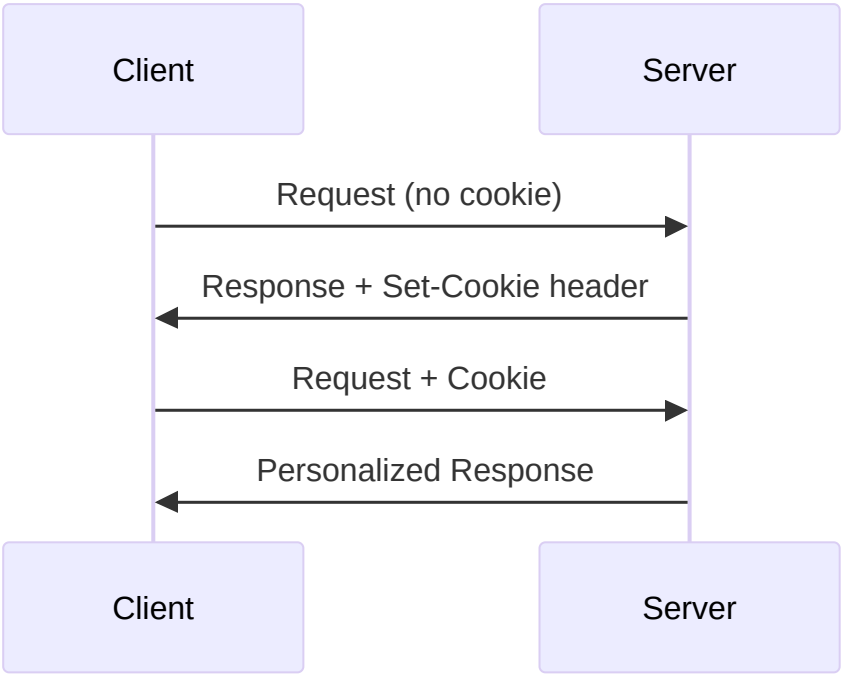
Creating a Cookie

```
Cookie cookie = new Cookie("username", "john");
cookie.setMaxAge(3600); // 1 hour
cookie.setPath("/");
response.addCookie(cookie);
```

Reading Cookies

```
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("username")) {
            String value = c.getValue();
            System.out.println("Username: " + value);
        }
    }
}
```

Visual: Cookie Flow



Key Takeaways

- ✅ Cookies stored on client side
- ✅ Can be disabled by users
- ✅ Limited size (4KB per cookie)

Topic 2.14: EJB Container Services

Overview

EJB containers provide a runtime environment for executing EJB components with various enterprise services.

Key Definitions

> ***EJB Container:*** *A runtime environment that manages EJB components, providing services like transaction management, security, and concurrency.*

Container Services

1. Lifecycle Management

EJB containers manage the lifecycle including:

- Instantiation
- Initialization
- Invocation
- Passivation
- Removal

They create instances as needed, pool and reuse instances, and destroy when no longer in use.

2. Transaction Management

EJB containers provide:

- **Declarative Transaction Management:** Using annotations or deployment descriptors
- **ACID Properties:**
  - Atomicity: All or nothing
  - Consistency: Valid state
  - Isolation: Concurrent transactions don't interfere
  - Durability: Committed data is permanent

```java
@Stateless
public class AccountService {

    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void transfer(Account from, Account to, BigDecimal amount) {
        from.withdraw(amount);
        to.deposit(amount);
    }
}
```

3. Security Services

EJB containers enforce:

- Role-based access control (RBAC)

- Authentication
- Authorization
- Encryption of communication

```
@Stateless
public class AccountService {

    @RolesAllowed({"customer", "admin"})
    public Account getAccount(int id) {
        return accountRepository.find(id);
    }
}
```

4. Concurrency Management

EJB containers manage:

- Thread synchronization
- Concurrency policies
- Instance pooling

They ensure thread safety and prevent race conditions.

5. Resource Management

EJB containers handle:

- **Database Connections:** Connection pooling
- **JMS Connections:** Message queue management
- **JNDI Resources:** Naming and directory services

6. Remote Method Invocation (RMI)

EJB containers provide:

- Remote method invocation support
- Marshalling/unmarshalling of parameters
- Network communication handling

7. Dependency Injection (DI)

EJB containers support dependency injection:

- Inject resources at runtime
- Improve modularity and testability

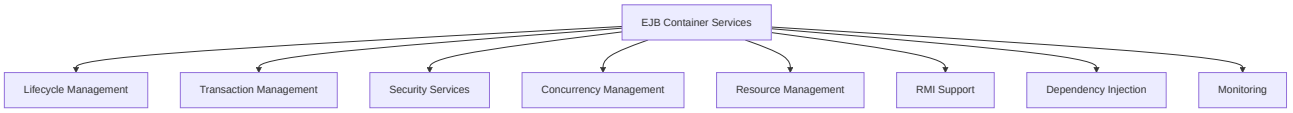
```
@Stateless
public class OrderService {

    @EJB
    private InventoryService inventoryService;

    @PersistenceContext
    private EntityManager em;

    @Resource
    private SessionContext ctx;
}
```

8. Monitoring and Management

EJB containers provide:

- Performance monitoring
- Health checking
- Usage tracking
- Management interfaces

Visual: EJB Container Services



Key Takeaways

- ✅ Container manages lifecycle automatically
- ✅ Declarative transaction management
- ✅ Role-based security
- ✅ Resource pooling and management

- ✅ Dependency injection support

📊 Coverage Statistics

Metric	Value
Total Syllabus Topics (Unit 1 & 2)	26
Topics Covered	26 (100%)
Unit 1 Coverage	12/12 (100%)
Unit 2 Coverage	14/14 (100%)

Notes compiled for Advanced Java Unit 1 and Unit 2. Complete coverage of all topics achieved including Mid Sem topics.